

CS6004NT

C# Introduction and Setup

About the Module

Module Information

Module Code: CS6004NT

Module Name: Application Development

Duration: Year Long (Semester 1 and Semester 2)

Credit: 30

Module Category: Core and Critical Module



Class Structure

Lecture: 24 Lectures (12 Lectures per Semester)

Workshop : Each lecture class is followed by 2 hours long workshop class

Content : All lectures and Workshops content will be uploaded in **MySecondTeacher** Platform.

Students' Performance Tracking: Workshop tracking, Weekly Assignment, Power Test

Assessment Details

Assessment Method	Description	% Weighting	Estimated Due Week
Coursework	Coursework 1 (Individual Coursework)	30	End of Semester 1 (Week 12)
Coursework	Coursework 2 (Group Coursework)	30	End of Semester 2 (Week 24)
Unseen Examination	Multiple Choice Question (MCQ)	40	Actual Date to be confirmed

Application Development

- Application development is the **process of creating a software or a set of softwares** to perform the different tasks that a business or individual requires.
- Applications such as **Microsoft Word, Excel, PUBG, and MySecondTeacher** are real-world examples that have been developed through this process.



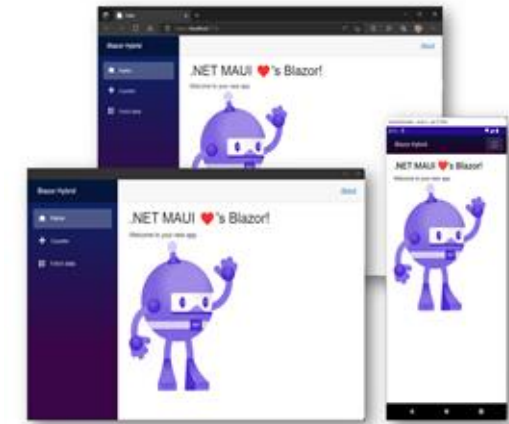
So, what is Application Development ?

Application Development

- **Hence**, in this module, we will learn how to **develop** applications by following the application development process, using the **C# programming language** and the **.NET framework**.
- You will learn both **Frontend** and **Backend** Software Development.

Overview of Contents (Semester 1)

- **C# Fundamentals:** Syntax, OOP, data structures, LINQ, async programming.
- **.NET Ecosystem:** SDK, MAUI (multi-platform app dev), Blazor (web & hybrid apps).
- **App Development:** Build console and Desktop applications.
- **Tools & Practices:** Docker, SQL/SQLite, dependency injection, debugging, and modern patterns (SOLID, DRY).
- **Blazor/MAUI:** Create cross-platform UIs, handle routing, forms, state management, and integrate with APIs.



Overview of Contents (Semester 2)

- **ASP.NET Core Fundamentals:** Understand the framework and how it powers modern web applications.
- **Building RESTful APIs:** Learn how to design and implement scalable APIs.
- **Complete Backend Development:** Hands-on with the .NET Web API template to create a full backend system.
- **Entity Framework Core:** Implement ORM to connect and interact with databases.
- **Secure Authentication & Authorization:** Integrate JWT (JSON Web Tokens) for secure user management.



Week 1

C# Introduction and Setup

Learning Objectives

- Explain the difference between C# and .NET
- Read simple C# code and explain its output
- Verify the SDK and run a first console project

1. Introduction To C# and .NET

1. C# language
2. .NET framework
3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

One lesson, three outcomes. Depth matters more than speed.

Introduction to C# & .NET

C# Language

1. Introduction To C# and .NET

1. C# language
 2. .NET framework
 3. History of C# & .NET
- ### 2. Why .NET stands out
- ### 3. .NET Runtimes
1. Required software (SDK, IDE, Workloads)
- ### 4. Run C# code
1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Introduction To C# & .NET

C# Language

- **C#** (pronounced "C - Sharp") is a **versatile, high-performance** programming language at the heart of the .NET ecosystem developed by **Microsoft**.
- The **syntax in C#** is quite similar with **Java**.



Microsoft

1. Introduction To C# and .NET

1. C# language

2. .NET framework

3. History of C# & .NET

2. Why .NET stands out

3. .NET Runtimes

1. Required software (SDK, IDE, Workloads)

4. Run C# code

1. Using Visual Studio

2. Using Rider

3. Using CLI

Introduction To C# & .NET

C# Language

```
public class Program
{
    // Main method with a print statement
    0 references
    public static void Main()
    {
        Console.WriteLine("Hello from C#");
    }
}
```

A Simple C# Program

1. Introduction To C# and .NET

1. C# language

2. .NET framework

3. History of C# & .NET

2. Why .NET stands out

3. .NET Runtimes

1. Required software (SDK, IDE, Workloads)

4. Run C# code

1. Using Visual Studio

2. Using Rider

3. Using CLI

Introduction To C# & .NET

C# Language

- Fully **open-source**, it empowers developers to build robust applications virtually across any platform – from **smartphones and desktops to cloud servers and IoT devices**.



1. Introduction To C# and .NET

1. C# language

2. .NET framework

3. History of C# & .NET

2. Why .NET stands out

3. .NET Runtimes

1. Required software (SDK, IDE, Workloads)

4. Run C# code

1. Using Visual Studio
2. Using Rider
3. Using CLI

Introduction To C# & .NET

C# features

Key Features of C#

- **Simple & Modern** → Clean, easy-to-learn syntax.
- **Type-Safe** → Strong compile-time & runtime checking.
- **Object-Oriented** → Classes, inheritance, polymorphism, encapsulation.
- **Cross-Platform** → Runs on Windows, Linux, macOS via .NET.
- **Automatic Memory Management** → Garbage Collection.
- **Generics** → Type-safe and reusable code.
- **LINQ** → SQL-like queries on collections & data.
- **Asynchronous Programming** → async/await for concurrency.
- **Rich Library Support** → .NET Base Class Library (BCL).

1. Introduction To C# and .NET

1. C# language

2. .NET framework

3. History of C# & .NET

2. Why .NET stands out

3. .NET Runtimes

1. Required software (SDK, IDE, Workloads)

4. Run C# code

1. Using Visual Studio

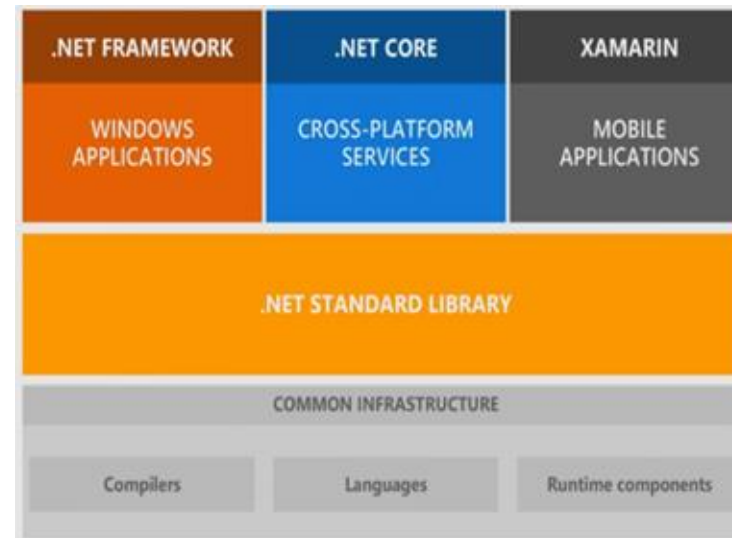
2. Using Rider

3. Using CLI

Introduction To C# & .NET

.Net

- **.NET** (pronounced as "**Dot NET**") isn't just a **framework**—it's a **free, open-source, cross-platform** ecosystem for building *anything, anywhere*. Born at **Microsoft** but now community-driven, it unifies development across **Windows, Linux, macOS, mobile, cloud, and IoT**.
- **C#** runs on **.NET platform**



1. Introduction To C# and .NET
 1. C# language
 2. **.NET framework**
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Introduction To C# & .NET

.Net

- **Supports Multiple Languages:** C#, F#, Visual Basic
- **Cross Platform:** Windows, Mac, Linux
- One consistent **API** among all **platforms**.
- **Libraries:** NuGet

1. Introduction To C# and .NET
 1. C# language
 2. **.NET framework**
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Introduction To C# & .NET

History

- In **2000**, Microsoft announced C# at the Professional Developers Conference. It is designed by **Anders Hejlsberg**.

C# Version	Release Date	.NET Version	Visual Studio
C# 1.0	Jan 2002	.NET Framework 1.0	Visual Studio .NET 2002
C# 2.0	Nov 2005	.NET Framework 2.0	Visual Studio 2005
C# 3.0	Nov 2007	.NET Framework 3.5	Visual Studio 2008
C# 4.0	Apr 2010	.NET Framework 4.0	Visual Studio 2010
-----	-----	-----	-----
C# 11	Nov 2022	.NET 7.0	Visual Studio 2022
C# 12	Nov 2023	.NET 8.0 (LTS)	Visual Studio 2022
C# 13	Nov 2024	.NET 9.0	Visual Studio 2022 (v17.12)
C# 14	Nov 2025	.NET 10.0 (LTS)	Visual Studio (2025)

1. Introduction To C# and .NET

1. C# language
2. .NET framework

3. History of C# & .NET

2. Why .NET stands out

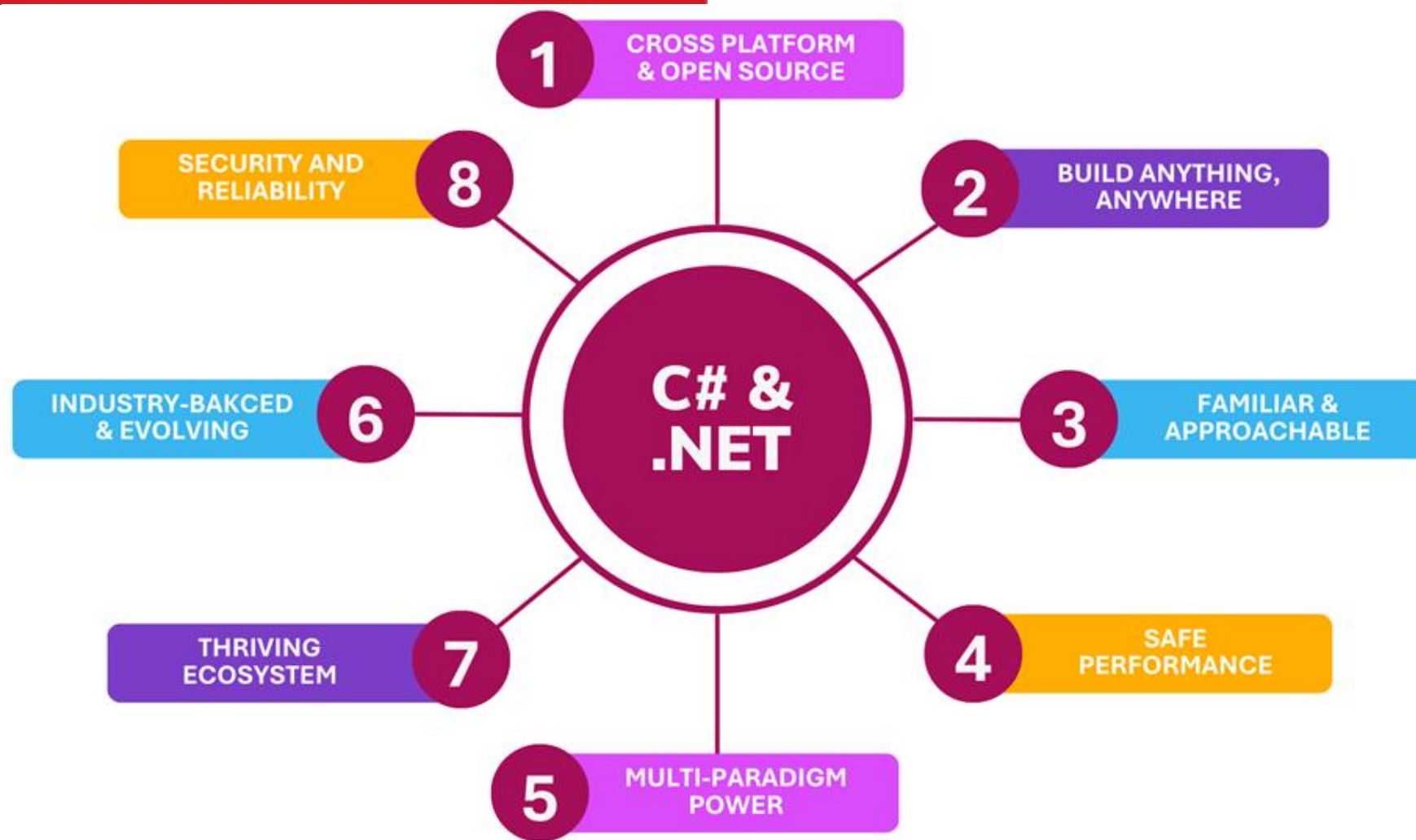
3. .NET Runtimes

1. Required software (SDK, IDE, Workloads)

4. Run C# code

1. Using Visual Studio
2. Using Rider
3. Using CLI

Why .NET Stands Out



1. Introduction To C# and .NET

1. C# language
2. .NET framework
3. History of C# & .NET

2. Why .NET stands out

3. .NET Runtimes

1. Required software (SDK, IDE, Workloads)

4. Run C# code

1. Using Visual Studio
2. Using Rider
3. Using CLI

Why .NET Stands Out

Universal & Open

Windows, Linux, macOS, mobile (iOS/Android), cloud (Azure/AWS), and IoT – all on the free, open-source .NET platform.

Build Anything, Anywhere

Create web apps, mobile apps (Xamarin/MAUI), games (Unity), desktop software (WinForms/WPF), and enterprise cloud services.

Familiar & Approachable

C-family syntax (like Java/C++) – uses `;`, `{}`, `if/else`, `for/while`, and `foreach` – with case-sensitive identifiers.

1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Why .NET Stands Out

Safe Performance

Delivers low-level languages efficiency *without* unsafe code in most scenarios (e.g., memory-safe high-performance APIs).

Multi-Paradigm Power

Object-oriented core + functional features (lambdas, LINQ) + modern async patterns.

Industry-Backed & Evolving

Powers 90% of .NET's runtime/libraries; updated yearly with cutting-edge features (e.g., records, pattern matching).

Thriving Ecosystem

Backed by Microsoft, 3.4M+ GitHub repositories, and tools like Visual Studio, VS Code, and

1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET

2. Why .NET stands out

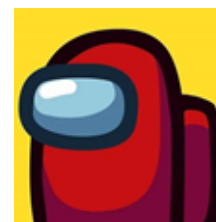
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Why .NET Stands Out

- Better career and salary opportunities



- Enterprise applications developed using .NET



1. Introduction To C# and .NET

1. C# language
2. .NET framework
3. History of C# & .NET

2. Why .NET stands out

3. .NET Runtimes

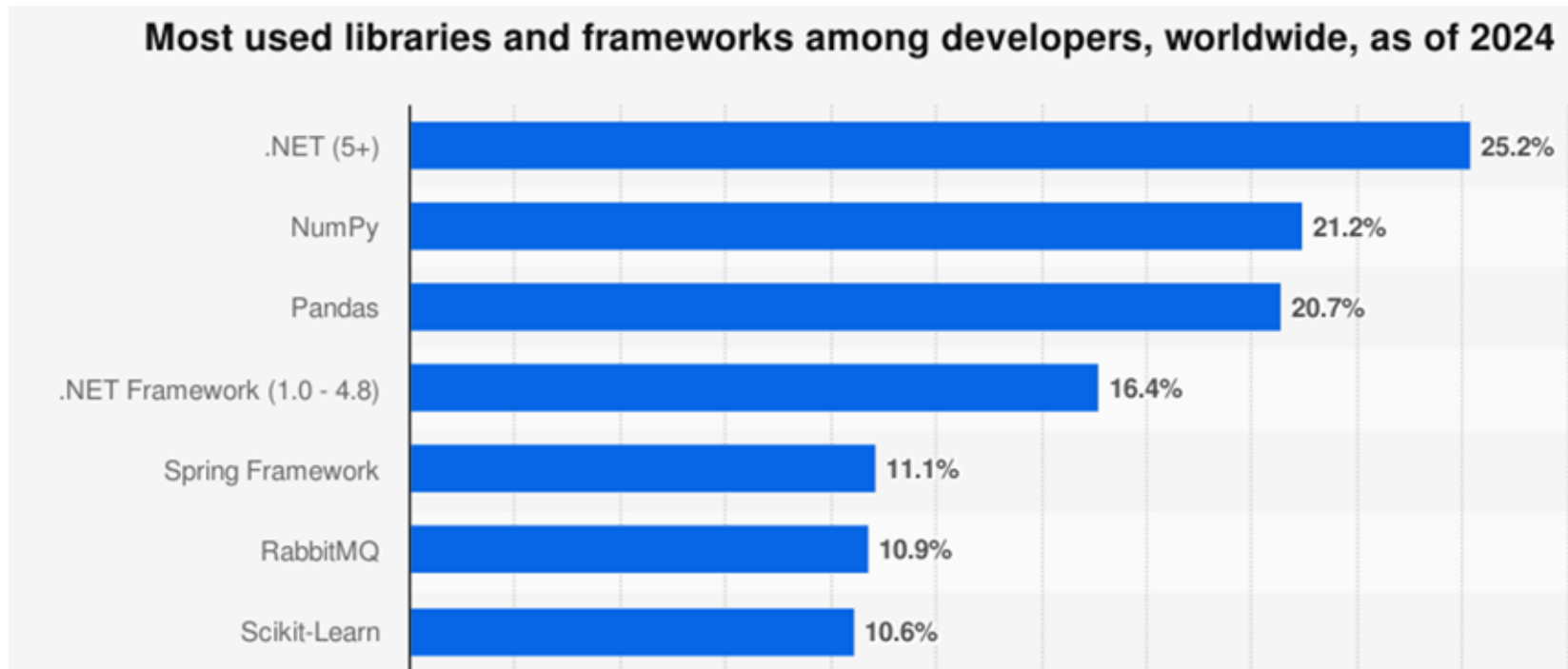
1. Required software (SDK, IDE, Workloads)

4. Run C# code

1. Using Visual Studio
2. Using Rider
3. Using CLI

Why .NET Stands Out

- Statistics of most used libraries by Statista.



1. Introduction To C# and .NET

1. C# language
2. .NET framework
3. History of C# & .NET

2. Why .NET stands out

3. .NET Runtimes

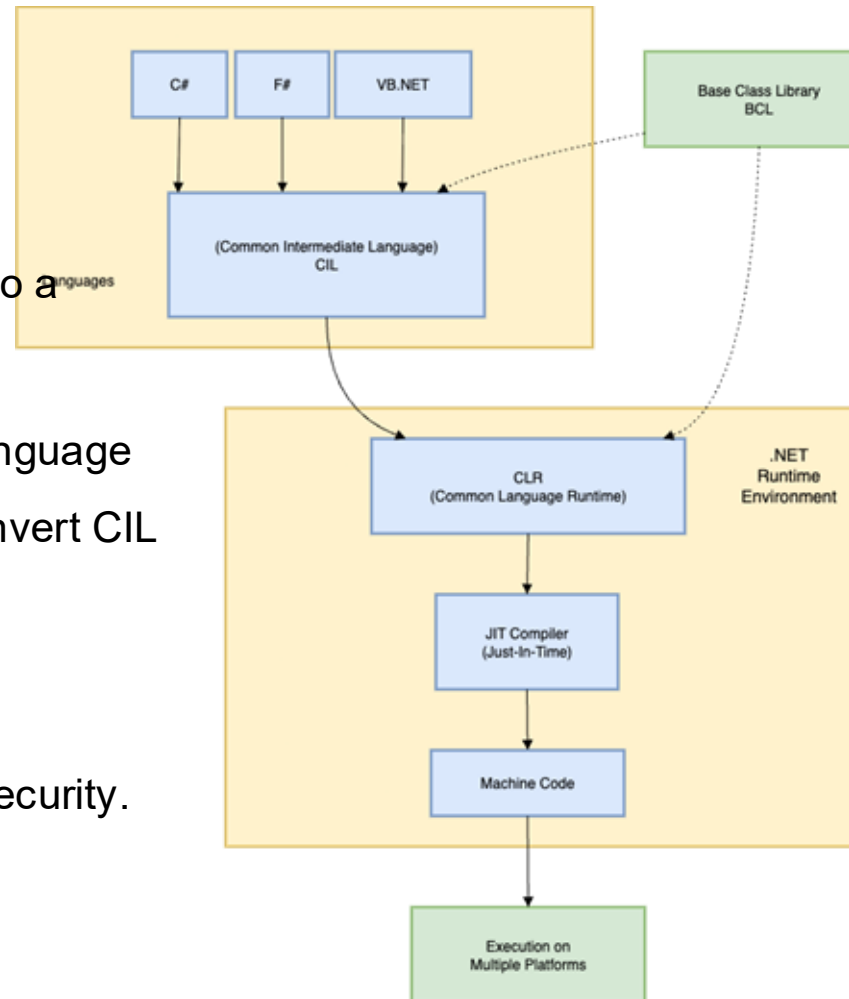
1. Required software (SDK, IDE, Workloads)

4. Run C# code

1. Using Visual Studio
2. Using Rider
3. Using CLI

.NET Runtime

- **Multi-Language:** Supports C#, F#, VB.NET.
- **Two-Step Compilation:** Code is first compiled to a universal **CIL** (Common Intermediate Language).
- **Runtime Compilation:** The **CLR** (Common Language Runtime) uses a **JIT** (Just-In-Time) compiler to convert CIL into native machine code at runtime.
- **Core Components:**
 - **CLR:** Manages execution, memory, and security.
 - **BCL:** Provides a huge library of pre-built functionality.
 - **JIT:** Optimizes and compiles code for the specific running machine.



1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. **.NET Runtimes**
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

- **Key Benefit:** This process enables true **cross-platform** execution.

.NET Runtime

PUT THESE IN ORDER

- A. runtime
- B. C# source code
- C. program output
- D. compiler
- E. assembly with CIL

- 1. Introduction To C# and .NET
 - 1. C# language
 - 2. .NET framework
 - 3. History of C# & .NET
- 2. Why .NET stands out
- 3. .NET Runtimes
 - 1. Required software (SDK, IDE, Workloads)
- 4. Run C# code
 - 1. Using Visual Studio
 - 2. Using Rider
 - 3. Using CLI

.NET Runtime

1. C# source code
2. Compiler checks and translates the code
3. Assembly stores CIL and metadata
4. .NET runtime loads and runs the program
5. The program produces output

Daily-life example: a written order passes through several roles before the finished meal reaches the table.

1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

.NET Runtime

- **Windows:**

- **.NET SDK:** Install from [.NET Website](#)
- **IDE:** [Visual Studio 2022](#)
- **Containerization:** [Docker](#)



- **Mac:**

- **.NET SDK:** Install from [.NET Website](#)
- **IDE:** [JetBrains Rider](#) (free community edition supports .NET/MAUI/Blazor)
- **Containerization:** [Docker](#)



1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes

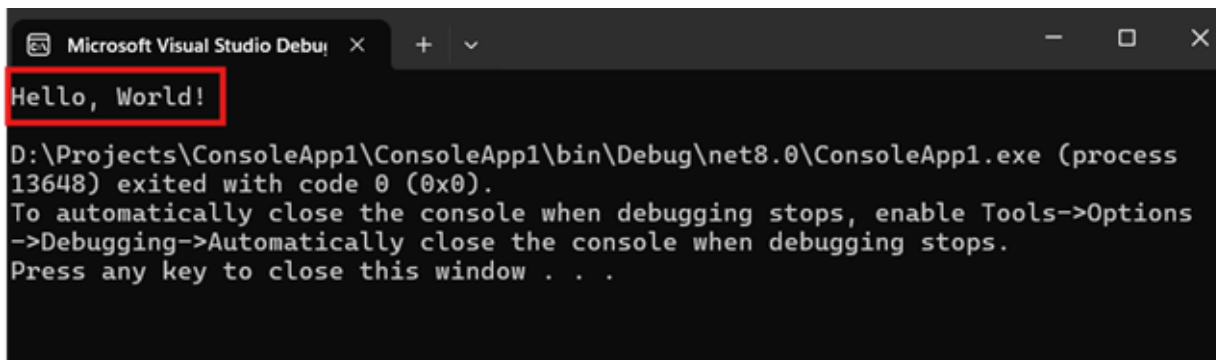
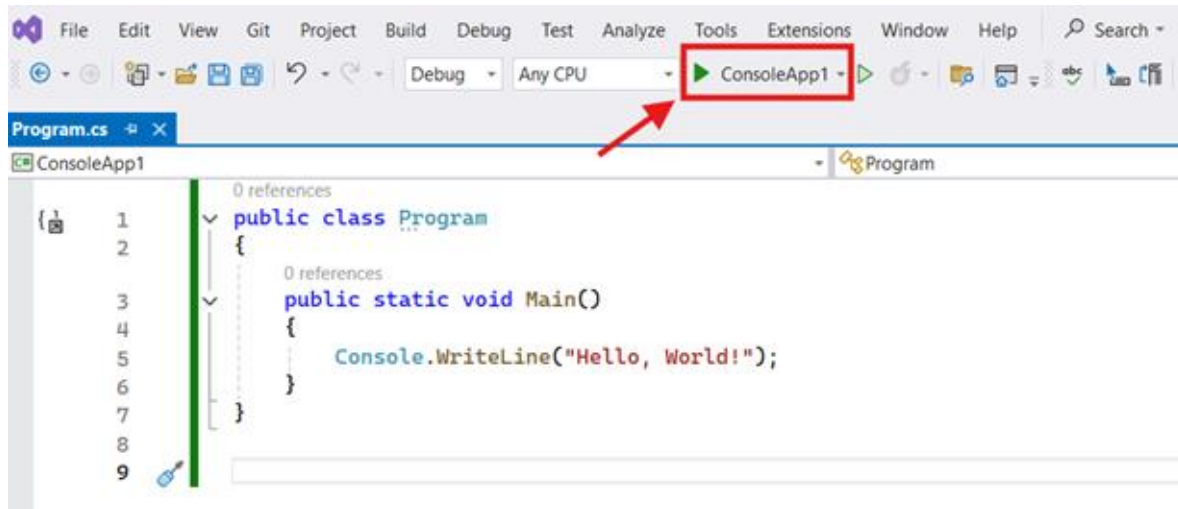
1. Required software (SDK, IDE, Workloads)

4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Note: Detail installation steps is Available on Workshop slides.

Run C# Code

Launch C# using Visual Studio



1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Run C# Code

Launch C# using Rider



/Users/aaryanjha/RiderProjects/WebApplication4/WebApplication4/bin/Debug/net9.0/WebApplication4

Hello World!

Process finished with exit code 0.

1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Run C# Code

Launch C# using CLI on Any OS

- *Edit Program.cs to write a program to print "**Hello World !**". You can use any text editor.*

```
Program.cs - Notepad
File Edit Format View Help

class Program
{
    public static void Main()
    {
        Console.WriteLine("Hello World!");
    }
}
```

1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Run C# Code

Launch C# using CLI on Any OS

- Create a new project using the command: ***dotnet new console -n <project_name>***
- Then go to the project folder by: ***cd /path/to/your/csharp-project***
- Check whether you have all the project files by running the command: ***ls or***

dir (Windows)

```
[aaryanjha@Aaryans-MacBook-Air RiderProjects % cd WebApplication4
```

```
aaryanjha@Aaryans-MacBook-Air RiderProjects % dotnet new console -n WebApplication4
The template "Console App" was created successfully.
[
Processing post-creation actions...
Restoring /Users/aaryanjha/RiderProjects/WebApplication4/WebApplication4.csproj:
Restore succeeded.
```

```
[aaryanjha@Aaryans-MacBook-Air WebApplication4 % ls
Program.cs          WebApplication4.csproj  obj
```

1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Run C# Code

Launch C# using CLI on Any OS

- Before running, we need to build the project: ***dotnet build***
- Then finally to run the application: ***dotnet run***

```
aaryanjha@Aaryans-MacBook-Air WebApplication4 % dotnet build
Restore complete (0.3s)
WebApplication4 succeeded (0.3s) → bin/Debug/net9.0/WebApplication4.dll

Build succeeded in 1.0s
aaryanjha@Aaryans-MacBook-Air WebApplication4 % dotnet run
Using launch settings from /Users/aaryanjha/RiderProjects/WebApplication4/WebApp
lication4/Properties/launchSettings.json...
Building...
Hello World!
aaryanjha@Aaryans-MacBook-Air WebApplication4 %
```

1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Run C# Code

Launch C# using CLI on Any OS

- Before running, we need to build the project: ***dotnet build***
- Then finally to run the application: ***dotnet run***

```
aaryanjha@Aaryans-MacBook-Air WebApplication4 % dotnet build
Restore complete (0.3s)
WebApplication4 succeeded (0.3s) → bin/Debug/net9.0/WebApplication4.dll

Build succeeded in 1.0s
aaryanjha@Aaryans-MacBook-Air WebApplication4 % dotnet run
Using launch settings from /Users/aaryanjha/RiderProjects/WebApplication4/WebApp
lication4/Properties/launchSettings.json...
Building...
Hello World!
aaryanjha@Aaryans-MacBook-Air WebApplication4 %
```

1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

First console project

One guided practical task

```
dotnet --version  
dotnet new console -n FirstApp  
cd FirstApp  
dotnet run
```

Evidence of success

- The terminal shows an SDK version
- The FirstApp folder contains a project
- The console displays the starter output

Think of the SDK as a toolbox. Today we use only the tools needed for one small working program.

1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Exit ticket

Question slide

ANSWER INDEPENDENTLY

- Describe the difference between C# and .NET in one sentence
- Order these stages: compiler, output, runtime, source code
- Write the command that creates a console project named FirstApp

1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

Complete all three. Check only after you have committed to your answers.

Exit ticket answers

Answer slide

SELF-CHECK

- C# is the programming language; .NET supplies the platform, runtime, libraries and SDK
- Source code, compiler, runtime, output
- `dotnet new console -n FirstApp`

1. Introduction To C# and .NET
 1. C# language
 2. .NET framework
 3. History of C# & .NET
2. Why .NET stands out
3. .NET Runtimes
 1. Required software (SDK, IDE, Workloads)
4. Run C# code
 1. Using Visual Studio
 2. Using Rider
 3. Using CLI

A correct answer matters. Being able to explain why it is correct matters even more.

Any Questions ?